Bank Campaign Decision Engine

Projeyi anlamak için kapsamlı teknik ve öğrenim raporu

Hazırlayan: Yaser
Tarih: 17 Ekim 2026
Kapsam: Aşama 1-8 ve doğrulama kayıtları

Her aşamada ne yaptık, neden yaptık ve sonuç bize ne söylüyor?

Bu rapor problem tanımından veri denetimine; modelleme, SQL, API, karar ekranı ve yerel teslim hazırlığına uzanan çalışmayı açıklar. Kodun çalışmasıyla modelin başarılı olması ayrı değerlendirilir. Gerçek sonuçlar ve sınırlılıklar birlikte sunulur.

Yayın sınırı: GitHub gönderimi ve halka açık dağıtım yapılmadı. Bu raporun hazırlanması için model yeniden eğitilmedi veya test kümesi yeniden skorlanmadı.

İçindekiler

1. Bu proje hangi problemi çözüyor? 3

2. Kullandığımız veri ve tahminin anlamı 4

3. Sistemin mimarisi: hangi parça ne işe yarıyor? 5

4. Aşama 1 - Ortamı ve proje sözleşmesini kurmak 6

5. Aşama 2 - Veriyi tanımak ve sızıntıyı önlemek 7

6. Aşama 3 - Baseline ve güvenilir değerlendirme düzeni 9

7. Aşama 4 - Model karşılaştırması ve karar motoru 11

8. Aşama 5 - PostgreSQL ve SQL özellik katmanı 13

9. Aşama 6 - FastAPI ile servisleme 14

10. Aşama 7 - Streamlit karar ekranı 15

11. Aşama 8 - Anlatılabilir ve taşınabilir teslim hazırlığı 17

12. Gerçek sonuçlar: model ne kadar iyi? 18

13. Bir adayın sistemdeki yolculuğu 20

14. Testler bize neyi kanıtlıyor? 21

15. Hangi dosyayı hangi soruyla okumalıyım? 22

16. Projeyi kendi başıma nasıl incelemeliyim? 23

17. Bu projenin açık sınırları ve sonraki araştırmalar 24

18. Sık sorulan sorular ve mülakatta anlatım 25

19. Kanıtlar, kaynaklar ve raporun sınırı 27

20. Teslim kontrol kaydı 28

Okuma önerisi: Önce 1-3 ile büyük resmi, 4-11 ile aşamaları okuyun. Sonra sonuçlar, istek yolculuğu, testler ve dosya rehberine geçin. Mülakat soruları kısa tekrar için uygundur.

1. Bu proje hangi problemi çözüyor?

Bir bankanın kampanya ekibinin elinde N aday kayıt, fakat yalnız K arama yapabilecek kapasite olduğunu düşünün. Herkesi aramak yerine hangi kayıtların önce ele alınacağına karar vermek gerekir. Proje, tarihsel vadeli mevduat aboneliği verisinden bir olasılık üretir, adayları bu olasılığa göre sıralar ve kapasite sınırı altında bir liste çıkarır.

Burada iki ayrı iş vardır. **Tahmin**, bir kaydın abonelik olasılığını hesaplamaktır. **Karar politikası**, bu olasılıklarla ve verilen kapasite/maliyet/değer varsayımlarıyla kimlerin listeye gireceğini belirlemektir. Aynı tahminler, farklı K veya C/V ayarlarıyla farklı listeler üretebilir. Bu yüzden projeyi yalnızca bir sınıflandırıcı olarak tasarlamadık.

Çıktı bir kredi onayı, müşteri uygunluğu veya otomatik banka kararı değildir. Tarihsel bir eğitim verisiyle kurulmuş çevrimdışı portföy prototipidir. İncelenen veri, yeni aramanın aboneliğe neden olup olmadığını ölçen rastgele bir deney içermediğinden, modelin çıktısı aramanın ek etkisini göstermez.

Başarıyı nasıl tanımladık? Sistemin girişlerini doğru doğrulaması, eğitim ve servis özelliklerinin tutarlı olması, tekrarlanabilir seçim yapması, sonuçların dürüstçe raporlanması ve bir kullanıcının aday CSV'sinden indirilebilir listeye ulaşabilmesi. Model başarısı ayrıca ölçüldü; yüksek yazılım kalitesinin yüksek model başarısıyla aynı şey olmadığı açık tutuldu.

Bir cümleyle proje

“Arama öncesinde mevcut olduğu varsayılan müşteri/geçmiş alanlarını kullanarak tarihsel abonelik olasılığı üreten, bu skorları kapasite ve beklenen değer politikasına dönüştüren; SQL, API, kullanıcı ekranı ve doğrulama kayıtları bulunan bir karar desteği prototipi geliştirdim.”

2. Kullandığımız veri ve tahminin anlamı

Veri, UCI Bank Marketing veri kümesinin **bank-additional-full.csv** sürümüdür. Daha eski bank-full sürümü veya yüzde 10'luk örnek dosya kullanılmadı. Dosyada 41.188 kayıt ve 21 sütun vardır: 20 kaynak girdi ve y hedefi. Tarihsel hedef, kişinin vadeli mevduata abone olup olmadığıdır; yes=1, no=0 olarak kodlanır.

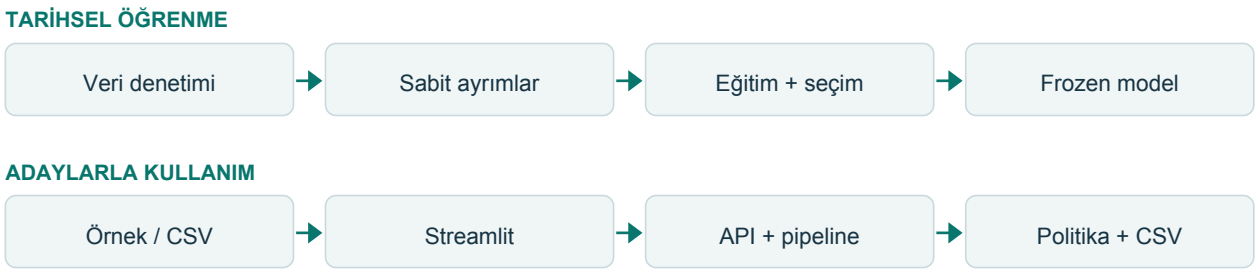
Gözlem	Kayıtlı değer	Anlamı
Toplam kayıt	41.188	Her satır doğrulanmış tekil bir müşteri sayılmaz
Abonelik yes	4.640	Azınlık hedef sınıf
Abonelik no	36.548	Çoğunluk hedef sınıf
Genel pozitif oran	%11,27	Genel veri denetimi; model performansı değil
İlk kopyadan sonraki tam tekrar	12	Ayrım aşamasında daha sonraki tekrarlar dışlandı
pdays=999	39.673	Belgelenmiş “önceden temas yok” işareti

Veri sözlüğü kaynak sırasının Mayıs 2008-Kasım 2010 dönemine göre olduğunu belirtir. Ancak dosyada tam tarih ve doğrulanmış müşteri kimliği bulunmaz. Dolayısıyla kesin gün bazlı bir zaman testi veya müşteri bazında tamamen ayrık bir değerlendirme yaptığımızı söyleyemeyiz.

Olasılık ile ek etki arasındaki fark

Modelin p=0,10 üretmesi, tarihsel veri üretim koşulları altında abonelik olasılığına dair bir tahmindir. “Bu kişiyi ararsak aboneliği 10 puan artırırız” anlamına gelmez. Zaten abone olmaya yatkın bir kişi, aranmasa da abone olabilir. Aramanın ek etkisi için temas edilen/edilmeyen grupları uygun bir deney düzeninde karşılaştırmak gerekir. Bu projede uplift modeli veya gerçek kampanya ROI ölçümü yapılmadı.

3. Sistemin mimarisi: hangi parça ne işe yarıyor?



PostgreSQL: ham kayıtlar + ayrımlar + dokuz özellik; Python ile eşitlik kontrolü.

Sistemi iki akış olarak düşünmek en kolay yoldur.

A. Tarihsel öğrenme ve değerlendirme akışı

UCI dosyası -> indirme/hash kontrolü -> şema ve veri denetimi -> sabit ayrımlar -> eğitimde öğrenilen ön işleme ve modeller -> doğrulamada karşılaştırma -> kararların dondurulması -> bir kez serbest bırakılan test değerlendirmesi -> kayıtlı model ve raporlar.

B. Yeni adaylarla kullanım akışı

Örnek/CSV -> Streamlit doğrulaması -> HTTP kampanya isteği -> FastAPI şeması -> yüklenmiş pipeline ile olasılık -> seçim politikası -> JSON yanıt -> ekranda liste ve CSV indirme.

PostgreSQL ham veriyi ve ayrımları saklar; SQL özellik görünümü aynı dokuz özelliği sunar. Mevcut eğitim Python veri yolunu kullanır. SQL katmanı sonradan eklendi ve Python ile birebir eşitliği doğrulandı; eğitim SQL'den yapıldı diye sunulmamalıdır. Benzer şekilde, kullanıcı CSV'sini skorlayan API'nin çalışması için veritabanının açık olması gerekmez.

Teknoloji	Bu projedeki somut görevi
Python / pandas	Dosyalar, tablolar, veri denetimi, politika ve rapor üretimi
scikit-learn	Pipeline, kodlama/ölçekleme, baseline ve metrikler
LightGBM	Sınırlı ağaç tabanlı model karşılaştırması
PostgreSQL / SQL	Ham veri, ayrım bilgisi, özellik ve etiket görünümleri
Pydantic / FastAPI	Sıkı HTTP sözleşmesi ve model servisleme
Streamlit	Aday yükleme, senaryo ayarı, sonuç görüntüleme ve indirme
pytest / Ruff / mypy	Davranış, kod standardı ve tip kontrolleri
uv / kilit dosyaları	Tekrarlanabilir Python bağımlılıkları
Docker / Compose / CI	Ortamın paketlenmesi ve otomatik kalite kontrolü

4. Aşama 1 - Ortamı ve proje sözleşmesini kurmak

Hangi sorunu çözüyorduk?

Kodun yalnızca geliştiricinin bilgisayarında tesadüfen çalışmasını önlemek ve PyCharm'ın hangi Python ortamını kullanacağını açık hale getirmek istedik. Ayrıca model yazmadan önce karar anını ve tahminin ne anlama geldiğini netleştirdik.

Ne yaptık ve niçin?

Python 3.12.11 ve proje içi .venv kullandık. Sanal ortam, bu projenin paketlerini diğer projelerden ayırır. İlk kurulumda görülen “No module named pip” sorunu, seçili yorumlayıcının ortamında pip bulunmadığını gösteriyordu; model koduyla ilgili değildi. Ortam araçları çalışır hale getirildi.

Proje src/bank_campaign altında kurulabilir bir paket yapısına taşındı. Böylece dosyaları rastgele çalışma dizininden veya PyCharm'ın farklı ayarlarından açmak yerine paket importlarıyla çalıştırabiliyoruz. pyproject.toml paket ve araç ayarlarını; uv.lock çözümlenmiş bağımlılık sürümlerini tutuyor. PyCharm yorumlayıcısı projenin .venv/bin/python dosyası olarak seçildi.

.gitignore; sanal ortamı, IDE dosyalarını, .env sırlarını, ham veriyi ve model artefaktlarını Git dışında tutuyor. Git init yalnızca yerel sürüm kontrolünü başlatır; GitHub'da repo oluşturmaz ve dosya yüklemez. İlk dal adıyla ilgili master/main mesajı bir hata değildi; yerel dal main olarak kullanılıyor.

Ne ürettik ve nasıl kontrol ettik?

Paket iskeleti, ortam ayarları, README, .env.example ve ilk veri/karar sözleşmesi üretildi. Paket importu ve modül çalıştırma kontrol edildi. Bu aşamada ortada tahmin davranışı olmadığı için model testi yazılmadı. Kazanım: hangi Python'ı çalıştırdığını, bağımlılığı ve sürüm kontrolü sınırlarını anlayabilmek.

5. Aşama 2 - Veriyi tanımak ve sızıntıyı önlemek

Neden önce model eğitmedik?

Bir sütunun hedefle güçlü ilişkisinin bulunması, karar anında kullanılabileceğini kanıtlamaz. Yanlış zamanda öğrenilen bir bilgiyle iyi görünen model, gerçek kullanımda aynı girdiyi bulamaz. Bu nedenle veri denetimi ve “arama öncesi neyi biliyoruz?” sorusu modelden önce geldi.

İndirme modülü resmî kaynağı alır ve yerel edinim kaydıyla SHA-256 dosya özeti tutar. Hash, dosyanın kaydettiğimiz sürüme göre değiştiğini anlamamızı sağlar; kaynak tarafından imzalanmış bir orijinallik belgesi değildir. Şema doğrulaması tam sütun kümesini, türleri, kategorileri, boşlukları ve sonlu sayıları kontrol eder. Dosyanın okunması ile doğru verinin okunması ayrı kontrollerdir.

Neden yalnızca dokuz özellik?

İhtiyatlı bir başlangıç sözleşmesi seçtik. Dahil edilen CRM/geçmiş alanlarının aramadan önce bilindiğini varsaydık. Veride anlık görüntü zamanları olmadığı için bu varsayımı bağımsız olarak kanıtlayamıyoruz.

Özellik	Ne anlatıyor?	Neden dahil?
age	Yaş	Arama öncesi müşteri bilgisi varsayımı
job	Meslek	Aynı CRM varsayımı
marital	Medeni durum	Aynı CRM varsayımı
education	Eğitim	Aynı CRM varsayımı; unknown korunur
default	Temerrüt durumu alanı	Arama öncesi kayıt varsayımı
housing	Konut kredisi alanı	Mevcut müşteri kaydı varsayımı
loan	Bireysel kredi alanı	Mevcut müşteri kaydı varsayımı
previous	Önceki kampanya temas sayısı	Mevcut kampanyadan önceki geçmiş
poutcome	Önceki kampanya sonucu	Önceki kampanya geçmişi

Bunların iki tanesi sayısal, yedi tanesi kategoriktir. Özellik listesi schema.py içindeki BASELINE_FEATURES ile merkezi olarak tanımlıdır. row_id modele verilmez; yalnız kayıtları eşlemek için kullanılır. y girdinin cevabıdır, özellik olamaz.

Hangi alanları neden çıkardık?

Alan/grup	Dışlama nedeni
duration	Aramanın süresi ancak arama bitince bilinir; doğrudan bilgi sızıntısı
contact, month, day_of_week	Kaydedilmiş son temasın bilgileri; planlanan aramanın doğrulanmış bilgisi değil
campaign	Mevcut son teması da içeren birikimli sayı
pdays	999 anlamlı bir işaret; karar anına göre kesin tarih doğrulanamıyor
Makro göstergeler	Yayım gecikmesi ve o gün bilinen sürüm/vintage doğrulanamıyor
y	Tahmin edilecek hedef; girdi değil

Her dışlanan alanın her uygulamada kullanılamaz olduğunu iddia etmiyoruz. Bu veri ve karar zamanı sözleşmesi altında kullanılabilirliğini gerekçelendiremediğimiz için başlangıç modeline almadık. Daha çok sütun kullanmak otomatik olarak daha doğru yöntem demek değildir.

Unknown ve tekrarlar

Dosyada boş hücre görülmemesi, her bilginin eksiksiz olması anlamına gelmez. Örneğin default alanında 8.597 kayıt unknown'dır. Bunları sıfıra çevirmek veya satırı atmak yerine anlamlı kategori olarak koruduk. pdays=999'u gerçek 999 gün gibi yorumlamadık.

Dokuz özellikte aynı profile sahip çok sayıda kayıt vardır. İki kişinin yaşı, mesleği ve kredi alanları aynı olabilir. Bu profilleri tek kişi sayıp birleştirmedik. Tam kaynak girdi tekrarlarını ise sonraki aşamada etiket kullanmadan kontrol ettik.

Çıktılar: data_audit.md/json, sütun kararları ve edinim kaydı. **Kazanım:** veri sızıntısı, şema, veri kaynağı, eksik bilgi ve kimlik varsayımlarını birbirinden ayırmak.

6. Aşama 3 - Baseline ve güvenilir değerlendirme düzeni

Eğitim, doğrulama, test neden ayrı?

Eğitim, modelin parametreleri öğrendiği bölümdür. Doğrulama, adayları karşılaştırıp seçim yaptığımız bölümdür. Test, bu kararlar verildikten sonra ayrılmış son kontrol bölümüdür. Test sonucuna bakıp modeli tekrar seçersek test de geliştirme sürecinin parçası olur; bağımsız son kontrol niteliği zayıflar.

Kaynak sırasını karıştırmadan yüzde 60/20/20 ayırdık. Bu, tarihsel sıralamayı tamamen rastgele karıştırmaktan kaçınır. Yine de tam tarihler olmadığı için eksiksiz bir zamansal değerlendirme iddiası değildir.

Bölüm	Atanan	Dışlanan tekrar	Kullanılan
Eğitim	24.712	7	24.705
Doğrulama	8.238	3	8.235
Test	8.238	2	8.236

Tam tekrar kontrolünde y dışındaki 20 kaynak alanı karşılaştırılır; en erken satır tutulur, sonraki aynılar dışlanır. duration gibi modele alınmayan alanların burada bulunması, onları tahmin özelliği yapmaz. Bu işlem, çevrimdışı veri temizliği içindir. Kontrol y'ye bakmadığından hedefi kullanarak satır seçmez. Satır kimliği kaynak dosyasındaki sıfır tabanlı konumdur ve silme/filtreleme sonrası yeniden numaralandırılmaz.

Baseline neden gerekli?

Bir modelin “iyi” olduğunu anlayabilmek için basit referanslara ihtiyacımız var. DummyClassifier prior, herkese eğitimdeki pozitif oranı verir. Yaş/geçmiş/kategori ilişkisini öğrenmez. Lojistik regresyon, özelliklerden düzenli ve anlaşılabilir bir olasılık modeli kuran ikinci referanstır.

Lojistik pipeline içinde age/previous StandardScaler ile ölçeklenir; kategoriler OneHotEncoder ile sayısal gösterime çevrilir. Örneğin job=retired için ayrı bir gösterge oluşabilir. Kodlayıcıya “kategori 1, 2, 3” diye keyfi büyüklük sırası öğretmeyiz. Eğitimde görülmemiş ama sözleşmede geçerli bir kategori, handle_unknown=ignore ile işlenir. Resmî sözlük dışındaki yazım hataları servis sınırında reddedilir; bu iki durum farklıdır.

Pipeline neden önemli? Ölçekleyici ve kodlayıcı yalnız eğitimde fit edilir. Doğrulama/test için yeniden öğrenilmez. Verinin tamamında ölçekleme veya kategori öğrenip sonra ayırmak, değerlendirme bilgisinin ön işlemeye sızmasına yol açabilir. Kaydedilen pipeline, hem ön işlemeyi hem modeli aynı nesnede taşır.

Lojistik ayarları sabitti: C=1, lbfgs, max_iter=1000. Bu aşamada sınıf ağırlığı, oversampling, kalibrasyon veya hiperparametre araması yapılmadı. Önce sade ve denetlenebilir bir başlangıç noktası kurduk.

Bu aşamanın sonuçları

Model	Doğrulama AP	ROC-AUC	Brier
Dummy prior	0,110747	0,500000	0,102408
Lojistik regresyon	0,148423	0,572422	0,101965

Lojistik model dummy'yi geçti, fakat ayrıştırma gücü sınırlıydı. Test bu aşamada skorlanmadı. Model, ön işleme ve doğrulama tahminleri kaydedildi; kaynak/ayrım/kod/bağımlılık özetleri raporlandı.

7. Aşama 4 - Model karşılaştırması ve karar motoru

Neden LightGBM ekledik?

Lojistik model iyi bir referans olsa da ilişkileri sınırlı bir biçimde temsil eder. Ağaç tabanlı boosting, doğrusal olmayan ilişkileri ve özellik etkileşimlerini yakalayabilir. Bunun gerçekten fayda sağlayıp sağlamadığını aynı ayrımlar ve aynı dokuz özellikle kontrol ettik. Model ailesini değiştirmek için veri sözleşmesini gevşetmedik.

Üç sınırlı aday önceden belirlendi: small (7 yaprak/100 ağaç), medium (15/100), long (15/200). Ortak ayarlar arasında learning_rate=0,05, min_child_samples=50, reg_lambda=1 ve seed=42 vardır. Deterministik ayarlar ve tek iş parçacığı kullanıldı. Yalnız eğitim verisinde fit edildi; erken durdurma veya doğrulamada yeniden fit yoktu.

Model	Doğrulama AP	ROC-AUC	Brier
Dummy prior	0,110747	0,500000	0,102408
Lojistik regresyon	0,148423	0,572422	0,101965
LightGBM small	0,156660	0,584898	0,101857
LightGBM medium	0,143710	0,561174	0,101831
LightGBM long	0,140416	0,554931	0,101813

Seçim kuralı: önce doğrulama Average Precision yüksekliği; eşitse düşük Brier; yine eşitse isim sırası. Bu kuralla **lightgbm_small** seçildi. Tek bir açık ölçüt, sonuçları gördükten sonra avantajlı ölçüt seçme esnekliğini azaltır.

Neden “en iyi model” ifadesini sınırlıyoruz?

AP kazananı, her arama kapasitesinde en çok abonelik bulan model değildir. Doğrulamada K=500 için medium 98, lojistik 97, seçilen small 89 tarihsel abonelik buldu. Buna rağmen hedef kuralımız AP olduğu için test sonucunu gördükten sonra modeli değiştirmedik. İş hedefi yalnız ilk 500 kişiye odaklanacaksa gelecekte yeni bir değerlendirme düzeninde seçim ölçütünü baştan buna göre tasarlamak gerekir.

Karar politikası nasıl çalışır?

K, en fazla seçilecek kayıt sayısıdır. C, seçilen kayıt başına varsayımsal maliyet; V, abonelik başına varsayımsal değerdir. C ve V aynı para biriminde olmalıdır. Para birimi ve gerçek maliyet veri kümesinden gelmez; kullanıcı varsayımıdır.

- **capacity:** Olasılığa göre azalan sıralayıp en fazla K kayıt seçer. Beklenen değeri negatif kayıtları da seçebilir.
- **positive_value:** Önce p × V - C > 0 olanları tutar, sonra aynı sıralamayla en fazla K seçer.
- Eşit olasılıkta küçük row_id önce gelir; aynı girdi aynı sıralamayı üretir.
- K=0 veya boş girdi boş sonuç verir. K>N ise aday sayısını aşmaz. Negatif/sonsuz/geçersiz sayılar kabul edilmez.

Elle hesaplanabilir örnek

K=2, C=3, V=100 olsun. Üç adayın olasılıkları A=0,20, B=0,05 ve C=0,01 ise beklenen net değerleri sırasıyla 17, 2 ve -2 olur. capacity ilk iki adayı seçer. positive_value da bu ayarda ilk ikiyi seçer. Maliyet 8'e çıkarsa değerler 12, -3, -7 olur; positive_value yalnız A'yı seçer. Kapasitenin dolmaması hata değildir.

Seçili kayıtlar için beklenen abonelik sum(p), varsayımsal maliyet seçilen_adet × C, beklenen net değer sum(p × V - C) olarak hesaplanır. Bir sonucun 0,11 abonelik çıkması, gerçek dünyada bir kişinin yüzde 11'i oluşacak demek değildir; tekrarlı benzer senaryolar altında ortalama beklentidir.

Basit kural ve rastgele referans niçin var?

Sabit geçmiş kuralı, önce önceki kampanyası başarılı olanları, sonra previous değeri yüksek olanları, sonra row_id'yi sıralar. Kural sonuçlar görülmeden tanımlandı. Rastgele referans ise 200 tekrar ve sabit seed ile üretildi. Böylece karmaşık modelin basit bir stratejiden gerçekten daha iyi olup olmadığını görebiliyoruz.

Dummy'nin eşit skorları row_id ile çözmesi onu rastgele kampanya yapmaz. Bu nedenle ayrı rastgele örnekleme referansı vardır. Rastgele listelerin yüzde 5/yüzde 95 değerleri, listeler arasındaki oynaklığı özetler; gelecekteki model başarısına ait güven aralığı değildir.

Modeli dondurmak ve testi açmak

Model, özellikler, senaryo, seçim kuralı ve ilgili dosya özetleri selection_freeze.json ile testten önce kaydedildi. Seçilen model eğitim+doğrulama üzerinde yeniden fit edilmedi. Son test raporu bu sabit kararların ayrı kontrolüdür. Testin tekrar tekrar yeni tahmin üretmesini önlemek için kilit, deneme kaydı, atomik tahmin checkpoint'i ve sonuç önbelleği kullanıldı.

Test sonrasında değerlendirme kontrol akışında bir uç durum onarıldı: testin daha önce açıldığını gösteren kayıt varken hem sonuç hem checkpoint kaybolursa otomatik yeniden skorlama engellendi. Bu onarım model/özellik/politika veya sonuçları değiştirmedi. Orijinal kaynak ve onarımın dosya özetleri ayrı tutuldu; sonradan değişmiş kod eski sonucu üretmiş gibi gösterilmedi.

8. Aşama 5 - PostgreSQL ve SQL özellik katmanı

Neden CSV'nin yanına veritabanı ekledik?

CSV öğrenme için yeterli olabilir, fakat gerçek veri işlerinde kalıcı saklama, anahtarlar, veri bütünlüğü ve ortak SQL sözleşmeleri önemlidir. Bu aşama, model geliştirmeye ek olarak veri mühendisliği yetkinliğini göstermek için yapıldı. Yeni yapay müşteri geçmişleri üretmedik; kaynakta olan veriyi güvenilir biçimde sakladık.

PostgreSQL 18.6 Docker ile çalıştırıldı. Şifre .env içinde yerel kaldı; .env.example yalnız örnektir. Bağlantı yerel adrese sınırlandı. Ham veri, veri yükleme kaydı ve ayrım atamaları ayrı tablolara kondu.

Nesne	Görev
campaign.dataset_loads	Kaynak ve ayrım özetleri, satır sayısı ve yükleme izi
campaign.raw_records	row_id ile bütün kaynak satırları
campaign.split_assignments	train/validation/test, dahil mi, tekrar ilişkisi
campaign.model_features	Yalnız row_id ve dokuz onaylı özellik
campaign.training_metadata	Hedef ve ayrım/dahil bilgisi; model özelliklerinden ayrı

model_features bir görünüm olduğu için yalnız belirlenen alanları dışarı açar; y/duration dahil değildir. training_metadata, CASE ile yes/no hedefini 1/0'a dönüştürür ve ayrımla birleştirir. Eğitim tüketicisi bu görünümleri kullanacaksa row_id ile birleştirip yalnız split='train' AND included kayıtlarını seçmelidir. Özellik görünümünde bütün 41.188 satırın bulunması, hepsinin eğitimde kullanılması demek değildir.

Idempotent ve transactional ne demek?

Idempotent yükleme, aynı veriyi ikinci kez yüklediğimizde satır sayısının artmamasıdır. Fakat yalnız ON CONFLICT DO NOTHING yazmak yeterli olmaz: mevcut satırın içeriği yanlışsa hatayı gizleyebilir. Yükleyici bu yüzden kaynak/ayrım sürümünü ve mevcut içeriğin doğrulanmış veriyle eşleşmesini de kontrol eder.

Transaction, işlemler grubunun birlikte başarılı olması veya geri alınmasıdır. Şema, staging tabloları, kopyalama, bütünlük kontrolleri ve görünümler tek işlemde ele alınır. Hata varsa yarım yükleme bırakılmaz. Advisory lock eşzamanlı yükleme çakışmalarını sınırlar.

Neden Python DB sürücüsü eklemedik?

Bu yerel aşamada mevcut Docker konteynerindeki psql istemcisini Python subprocess üzerinden kullandık. Böylece dondurulmuş model bağımlılıklarını değiştirmedik. Bu bilinçli bir yerel geliştirme tercihidir; üretim uygulamasında bağlantı havuzu olan ayrı bir sürücü seçimi başka bir tasarım işi olur.

Bütün ham alanlar ve dokuz özellik Python referansıyla karşılaştırıldı. İkinci yüklemede satır ve provenance kaydı korundu; hata halinde rollback test edildi. Entegrasyon testleri gerçek PostgreSQL üzerinde rastgele isimli ayrı test veritabanları açar ve yalnız onları temizler. Mevcut çalışma veritabanını sıfırlamaz.

9. Aşama 6 - FastAPI ile servisleme

Model dosyası neden tek başına yeterli değil?

Bir pipeline'ı notebook'ta çağırabilmek ile başka bir uygulamanın onu güvenilir kullanması farklıdır. Servisin hangi alanları kabul ettiği, ne döndürdüğü, model yoksa ne yaptığı ve sürümün nasıl izleneceği belli olmalıdır. FastAPI bu HTTP sınırını kurar; Pydantic ise veri sözleşmesini uygular.

Uç	Amaç	Önemli davranış
GET /health	Model hazır mı?	Hazır 200; yüklenememiş 503
GET /model-info	Model sürümü ve özellikler	Dosya özeti ve sınırlılıklar da döner
POST /score	Adayları skorlamak	Giriş sırasını korur
POST /campaign/select	Skor ve politika ile liste	Olasılık azalan, row_id artan sıra

Model, uygulamanın lifespan başlangıcında bir kez yüklenir; her istekte diskten okunmaz veya eğitilmez. Dışarıdan model dosyası yükleme ucu yoktur. İstekle gelen tekil teknik kimlik ile dokuz özellik, dokuz özelliklik pipeline girişine dönüştürülür; y veya duration gönderilirse 422 yanıtı verilir.

Eksik alanlar, yanlış kategori, fazla alan, sayısal string/bool, yinelenen row_id ve sonlu olmayan sayılar reddedilir. İstek başına en fazla 10.000 kayıt vardır. Yaş 1-120; row_id/previous güvenli tamsayı aralığında; kapasite 0-10.000; maliyet/değer sonlu ve 0-10^12 aralığındadır. Bu sınırlar bu prototipin sözleşmesidir; genel bankacılık kuralları olarak düşünülmemelidir.

422 ve 503 neden farklı?

422, isteğin sözleşmeye uymadığını anlatır; kullanıcı veriyi düzeltmelidir. 503, modelin veya tahmin servisinin hazır olmadığını anlatır; servis ortamı düzeltilmelidir. İkisine aynı genel hata mesajını vermek, yanlış tarafın sorunu çözmeye çalışmasına neden olur.

Model yükleyici pickle açmadan önce dosyanın tam frozen SHA eşleşmesini, metadata özetini, özellik sırasını ve kayıtlı Python/ML paket sürümlerini kontrol eder. Aynı doğrulanmış baytlar açılır. Komşu bir restoration kaydının değişmiş model baytlarını yetkilendirmesine izin verilmez. Ancak bu bir dijital imza sistemi değildir: hem manifest hem model güvenilmeyen bir kaynaktan gelirse pickle güvenli hale gelmez. Yalnız bu güvenilir yerel iş akışının dosyaları yüklenir.

API için ayrı requirements-api.lock kullanıldı; modelin frozen uv.lock dosyası değiştirilmedi. Bu ayrım, sonradan eklenen servis bileşeniyle önceki deney ortamının kaydını birbirine karıştırmamayı sağlar.

Kontrol: hazır/hazır değil yanıtları, modelin bir kez yüklenmesi, sıkı girdiler, deterministik skorlar, politika uç durumları ve hash kontrolünün pickle'dan önce yapılması test edildi. İlk üç SQL eğitim kaydının API skoru doğrudan pipeline çıktısıyla eşleşti; test kümesine yeni skor üretilmedi.

10. Aşama 7 - Streamlit karar ekranı

API varken ekran neden gerekli?

API geliştirici için uygundur, fakat kampanya senaryosunu denemek isteyen kişi JSON hazırlamak zorunda kalmamalıdır. Streamlit, mevcut hesaplamayı yeni bir arayüzden kullanılabilir yaptı. Modeli veya seçim hesabını ekranda yeniden yazmadık; ekran API'ye HTTP isteği gönderir.

Akış: örnek aday veya CSV seç -> adayları önizle -> K/C/V ve politikayı ayarla -> listeyi oluştur -> sonuçları ve varsayımsal özeti incele -> CSV indir. Yerel örnekler sentetiktir; gerçek müşteri diye sunulmaz.

CSV UTF-8/BOM ve virgül ayırıcı kullanır; en fazla 2 MiB/10.000 satırdır. Beklenen on sütun row_id ve dokuz özelliktir. Eksik/fazla/yinelenen sütunlar, geçersiz kategoriler, yinelenen kimlik ve yanlış tamsayılar reddedilir. Sayısal sütunları CSV'den bilinçli biçimde dönüştürüyoruz; y veya duration'ı sessizce düşürmüyoruz.

Neden eski sonucu temizliyoruz?

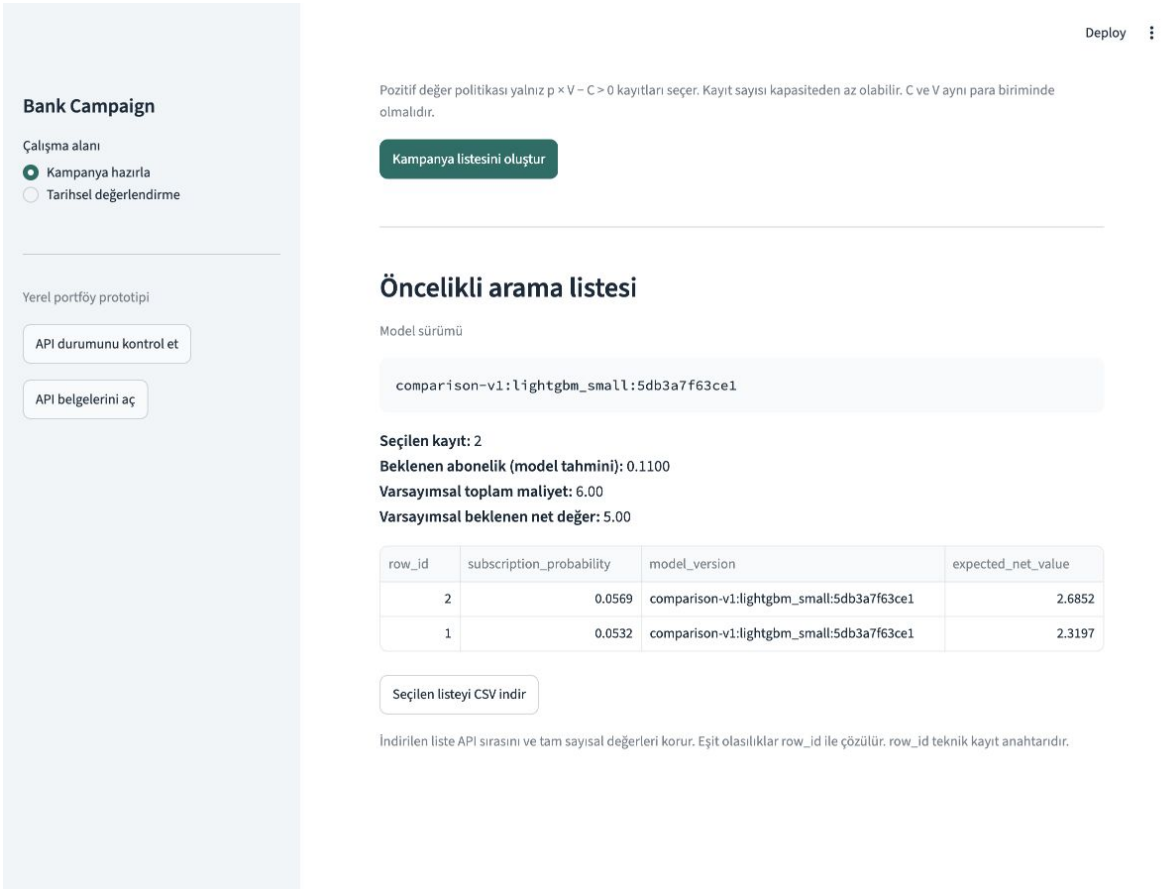
K=500 ile üretilmiş liste ekranda dururken K=100'e geçildiğini düşünün. Eski indir düğmesi kalırsa kullanıcı gördüğü ayarlarla üretilmemiş dosyayı indirebilir. Bu nedenle normalize kayıtlar, kaynak, API adresi ve senaryo ayarlarının özeti tutulur; değişiklikte önceki sonuç ve indirme kaldırılır. Başarısız yeni istekte de eski başarılı sonucu güncelmiş gibi göstermiyoruz.

İndirilen CSV, API'nin sıra ve tam sayısal değerlerini korur. Ekrandaki dört basamaklı olasılık gösterimi, CSV'nin hassasiyetini azaltmaz. Model sürümü de dosyada bulunur. Metin alanında tablo formülü başlatabilecek önekler dışa aktarma sırasında güvenli hale getirilir.

Tarihsel rapor neden ayrı?

Etiketsiz aday listesinde gerçek abonelik sonucunu bilmiyoruz. Bu nedenle yeni adaylar için precision/lift veya gözlenen başarı gösteremeyiz. Tarihsel değerlendirme görünümü, önceden kaydedilmiş doğrulama/test raporunu okur; API kapalıyken de açılır. Yeni kapasite ayarları bu eski raporu yeniden hesaplamaz.

Sekiz Streamlit AppTest akışı örnek/CSV, politika değişimi, eski sonuçların temizlenmesi, hatalı yükleme ve bağlantı hatalarını kontrol eder. Yardımcı katman için 22 test vardır. Ayrıca gerçek tarayıcıdan indirilen CSV, gerçek API yanıtıyla birebir karşılaştırıldı. Masaüstü ve 390 piksel genişlikte ekran kontrol edildi; dar ekranda politika ve özet okunabilirliği düzeltildi.



Gerçek yerel ekranda sentetik adaylarla üretilen kampanya listesi.

11. Aşama 8 - Anlatılabilir ve taşınabilir teslim hazırlığı

Bu aşamanın amacı

Çalışan kodu başka bir ortamda kurulabilir, incelenebilir ve anlaşılabilir hale getirmek. Bu rapor, “hangi aşamada neyi neden yaptık?” sorusuna cevap veren öğrenme katmanıdır. Docker, Compose, Makefile ve CI ise çalıştırma/doğrulama katmanıdır. GitHub'a gönderim bunlardan ayrı, daha sonraki bir işlemdir.

Dockerfile uygulama kodunu ve kilitli bağımlılıkları bir imajda toplar. Compose servislerin komutlarını, ağını ve sağlık kontrollerini tanımlar. Yerel .env sırları, ham veri ve pickle imajın içine alınmaz; güvenilir yerel model dosyası yalnız çalışma anında salt okunur bağlanır. Model dosyasının Git dışında olması bilinçlidir: temiz checkout tek başına otomatik hazır model içermez.

macOS üzerinde çözümlenmiş UI kilidi Linux için zorunlu watchdog paketini içermediğinden ilk konteyner kontrolünde bir platform farkı görüldü. Mevcut ML/API/UI kilitleri korunarak requirements-container.lock içinde Linux eki oluşturuldu; imaj ve CI bu eki de kurar ve paket uyumunu kontrol eder. Yalnız bağımlılık çözümünü atlayıp çalışıyor gibi göstermek yerine bildirilen bağımlılıklar tamamlandı.

Makefile sık kullanılan kurulum, kontrol, başlatma ve smoke adımlarına kısa isimler verir. CI dosyası, ileride bir uzak depoya gönderildiğinde test/lint/tip kontrollerinin temiz ortamda çalışmasını tanımlar. Yerelde testlerin geçmesi, GitHub Actions'ın uzak sunucuda çalıştırıldığı anlamına gelmez. Bu raporun teslim kontrol kaydı, hangisinin yerelde doğrulandığını açıkça belirtir.

Dondurulmuş model ile temiz kurulum arasındaki önemli ayrım

Aynı eğitim verisi ve algoritmayla yeniden üretilen bir pickle'ın baytları, ortam veya serileştirme farkı nedeniyle değişebilir. Çevrimdışı restoration yolu, doğrulama tahminlerinin eşitliğini kontrol ederek deneyin yeniden kurulmasına yardımcı olur. Servis ise mevcut güvenilir frozen pickle'ın tam dosya özetini ister. Dolayısıyla “restore komutunu çalıştırırsan her ortamda API kesin hazır olur” sözü verilmez. Mevcut exact-hash bundle korunmalı; farklı bir model sürümü ayrı bir sürümleme/değerlendirme işi olarak ele alınmalıdır.

Bu aşamada test sonucu iyileştirmek için tekrar eğitim, yeni özellik veya kalibrasyon eklenmedi. Yerel teslim doğrulamasının güncel ayrıntıları raporun sonundaki teslim kaydına eklenir. GitHub portföy yayını, bu raporun incelenmesinden sonraki ayrı adımdır.

12. Gerçek sonuçlar: model ne kadar iyi?

Önce metrikleri anlayalım

Average Precision (AP): Modelin pozitif kayıtları sıralamada öne taşımasını precision-recall ilişkisi boyunca özetler. Projede ana PR ölçütü non-interpolated Average Precision'dır; herhangi bir trapez PR alanıyla aynı hesap olduğu varsayılmaz. Daha yüksek iyidir; sınıf oranından etkilenir.

ROC-AUC: Rastgele bir pozitifin rastgele bir negatiften daha yüksek skor alma eğilimini özetler. 0,5 rastgele ayrıştırma düzeyidir. 0,525 sonucu “yüzde 52,5 müşteri doğru tahmin edildi” demek değildir.

Brier: Olasılık ile 0/1 sonuç farkının karelerinin ortalamasıdır. Daha düşük iyidir. Hem olasılık kalitesini hem veri dağılımını dikkate alan bir hata ölçüsüdür; tek başına kalibrasyon kanıtı sayılmaz.

Precision@K: Seçilen listede gözlenen abonelik sayısı / gerçekten seçilen kayıt sayısı. **Lift@K:** Bu precision'ın değerlendirme bölümünün genel pozitif oranına bölünmesi. Boş seçimde precision, sıfır pozitifli evrende lift tanımsız bırakılır; keyfi sıfır yazılmaz.

Accuracy ana ölçüt yapılmadı. Bütün veri için herkese no demek yaklaşık %88,73 accuracy verirken hiçbir aboneliği yakalayamaz. Bu örnek sınıf dengesizliğinde accuracy'nin neden tek başına yanıltıcı olabileceğini gösterir; test başarısı olarak kullanılmaz.

Son test sonuçları

Ölçüm	Seçilen model	Dummy referans
AP	0,341664	0,308281
ROC-AUC	0,525047	0,500000
Brier	0,278537	0,280944

Testte 8.236 kayıt vardır. K=500 için seçilen modelin tarihsel abonelik sayısı 231, precision'ı %46,20, lift'i yaklaşık 1,499'dur. Rastgele 200 listenin ortalaması 154,065; sabit geçmiş kuralının sonucu 372'dir. Kuralın precision'ı %74,40'tır.

Yöntem	K=500 listesinde tarihsel abonelik	Nasıl yorumlanmalı?
Rastgele	154,065 ortalama	Tek bir listenin sonucu veya güven aralığı değil
Seçilen ML modeli	231	Rastgeleden yüksek; sabit kuraldan düşük
Sabit geçmiş kuralı	372	Bu kontrol bölümünde modelden daha iyi

Dürüst sonuç: Bu model, bu test ve kapasitede basit geçmiş kuralını geçemedi. Projeyi, güçlü ve kullanıma hazır bir banka tahmin modeli diye sunmak doğru olmaz. Projenin değeri, başarısız veya sınırlı sonucu saklamadan uçtan uca mühendislik ve değerlendirme disiplinini gösterebilmesidir.

AP testte artmışken neden model daha iyi demiyoruz?

Eğitimde pozitif oran yaklaşık %4,81; doğrulamada %11,07; testte %30,83'tür. Testin taban oranı çok daha yüksektir. AP'nin mutlak değerini bu oranları göz ardı ederek dönemler arasında karşılaştırmak yanıltır. Test AP'sinin dummy referansının da 0,308281'e yükselmesi bunu görünür kılar. ROC-AUC'nin 0,525 olması ayrıştırmanın zayıf kaldığını gösterir.

Kalibrasyon sorunu neden önemli?

Testte seçilen 500 kayıt için modelin olasılık toplamı yalnız **39,67** beklenen aboneliktir; aynı tarihsel kayıtlarda gözlenen abonelik **231**'dir. Tahminler bu bölümde ciddi düşük kalmıştır. Bu fark, olasılık ve para senaryolarının güvenilirliğini sınırlar. “Sıralama biraz işe yarıyor” ile “p değeri doğru olasılık” aynı iddia değildir.

K=500, C=3, V=100 için varsayımsal maliyet 1.500, model tabanlı beklenen net değer 2.467,30'dur. Bunlar veri içindeki gerçek maliyet veya gelirlerden ölçülmemiştir. Gözlenen 231'i V ile çarpıp aramanın kazandırdığı gerçek kâr ilan etmek de nedensellik sorununu çözmez.

13. Bir adayın sistemdeki yolculuğu

Örnek dosyadaki row_id=2 kaydı: age=58, job=retired, marital=married, education=high.school, default=no, housing=no, loan=no, previous=1, poutcome=success.

1. Streamlit bu alanları örnek JSON'dan veya CSV'den alır. Dosya şeması ve türleri doğrulanır.
2. Kullanıcının K/C/V/mod ayarlarıyla CampaignRequest oluşur. HTTP ile /campaign/select çağrılır.
3. FastAPI aynı dokuz özellik sözleşmesini tekrar doğrular. UI kontrolü servis kontrolünün yerine geçmez; başka bir istemci doğrudan API'yi çağırabilir.
4. Daha önce yüklenmiş pipeline'a row_id hariç dokuz alan verilir. Eğitimde öğrenilmiş ön işleme tekrar kullanılır.
5. Frozen model bu sentetik kayda yaklaşık 0,0568518765 olasılık üretir.
6. C=3/V=100 için bu kaydın varsayımsal beklenen net değeri 2,6851876517 olur.
7. Diğer iki sentetik kaydın skorlarıyla birlikte sıralanır. K=2 için row_id=2 ve ardından row_id=1 seçilir.
8. Yanıt, kayıt başına skor/değer ve model_version; toplamda seçilen adet, beklenen abonelik, maliyet ve net değeri içerir.
9. Ekran sonucu gösterir ve API'nin sırasını/değerlerini taşıyan CSV sunar. Model sürümü comparison-v1:lightgbm_small:5db3a7f63ce1'dir.

Bu yolculukta SQL'den hedef etiketi sorgulanmaz, test verisi açılmaz, model eğitilmez. Üç sentetik kaydın toplam beklenen aboneliği yaklaşık 0,1100, varsayımsal maliyeti 6 ve beklenen net değeri 5,00 olan iki kayıtlık liste üretilir.

14. Testler bize neyi kanıtlıyor?

Aşama 7 sonunda PostgreSQL dahil 268 test geçti. Bu sayı sistemin model başarısını ispatlamaz; tanımlı davranışları, hata yollarını ve parçalar arasındaki sözleşmeleri kontrol eder. Aşama 8 sonundaki yeni doğrulama durumu ayrıca teslim kaydında belirtilir.

Test alanı	Kontrol ettiği risk
Veri doğrulama	Yanlış dosya/sütun/kategori/sayıyla sessiz devam etmek
Ayrım ve eğitim	Hedef sızıntısı, tekrar geçişi, yanlış fit bölümü
Model karşılaştırma	Config/seçim kuralı ve deterministik davranış
Politika	K=0, K>N, eşit skor, negatif/NaN ve boş liste
Test serbest bırakma	Testin yanlışlıkla tekrar skorlanması
SQL entegrasyonu	Çift yükleme, bozuk içerik, rollback, özellik eşitsizliği
Model yükleyici	Hash uyuşmadan pickle açılması, uyumsuz paket/özellik
API	422/503 ayrımı, sıkı şema, yükleme yaşam döngüsü
CSV ve ekran	Hatalı dosya, eski sonuç indirme, parametre değişimi
Gerçek smoke	Çalışan servisin örnek girdiden doğru biçimli çıktı vermesi

Ruff import ve kod standardı hatalarını; mypy işaretlenmiş tip sözleşmelerini kontrol eder. Bunlar davranış testlerinin yerine geçmez. PostgreSQL testleri gerçek veritabanına gider; API/ekran testlerinin önemli kısmı sentetik küçük fixture'lar kullanır. Böylece her kontrolde 41 bin satırı indirip gerçek modeli eğitmek gerekmez.

Bilinen test uyarısı, Starlette TestClient'ın mevcut HTTPX kullanımı için deprecation bildirimidir. Testler geçmektedir. Bu uyarı model performansını veya kullanıcı akışını değiştirmez; bağımlılık yenileme işi yeni sürümleme kapsamında ele alınabilir.

15. Hangi dosyayı hangi soruyla okumalıyım?

Dosya veya klasör	Sorulacak soru
docs/data_contract.md	Karar anı ve giriş/çıktı sınırımız ne?
data/schema.py	Tam kaynak şeması ve izinli özellik/kategoriler ne?
data/download.py, validate.py, audit.py	Veri nasıl ediniliyor, doğrulanıyor ve tanınıyor?
data/split.py	Satırlar bölümlere ve tekrarlara nasıl atanıyor?
features.py	Eğitimde ne öğrenilen ön işleme var?
train.py	Baseline nasıl eğitiliyor; hangi veriye dokunuluyor?
comparison.py ve configs/comparison.json	Adaylar ve model seçimi nasıl sabitlenmiş?
policy.py	Skordan arama listesine nasıl geçiliyor?
holdout.py	Freeze, restore, checkpoint ve tek test sınırı nasıl korunuyor?
sql/ ve data/database.py	DB içeriği ve SQL/Python eşitliği nasıl doğrulanıyor?
schemas.py	API hangi JSON'u kabul veya reddeder?
predict.py	Model hangi kanıtlarla yükleniyor ve skorlanıyor?
api.py	HTTP isteği hangi işlevlere yönleniyor?
dashboard.py ve app/streamlit_app.py	CSV, HTTP istemcisi, durum ve ekran nasıl bağlanıyor?
tests/	Her kuralın çalıştığı hangi örnekle gösteriliyor?
reports/	Sayısal bulgular ve kaynak özetleri nerede?

Tablodaki Python modülleri, aksi belirtilmedikçe src/bank_campaign altında bulunur. Ham veri ve artefaktlar Git dışında; raporlar ile kod sürüm kontrolüne uygundur. .env parola içerir; paylaşılacak dosya .env.example'dır.

SHA-256 ve manifest terimlerini nasıl okumalıyım?

Artefakt, eğitim sonunda kaydedilen pipeline gibi üretilmiş dosyadır. Manifest, bir çalışmanın hangi veri/kod/paket/çıktı sürümleriyle ilişkili olduğunu kaydeden listedir. SHA-256, dosya içeriğine bağlı bir özettir. Dosyadaki tek bayt değişikliği farklı özet üretir. Bu mekanizma “bu dosya aynı mı?” sorusuna cevap verir; “bu model bankada iyi sonuç verir mi?” sorusuna cevap vermez.

16. Projeyi kendi başıma nasıl incelemeliyim?

İlk turda modeli yeniden eğitmeyin. Çalışan ekranı kullanın, üç sentetik kayıtla K=2 ve K=1 senaryolarını deneyin. C=8 ve positive_value ile boş listeyi görün. CSV'yi indirip satır sırasını inceleyin. Ardından Swagger'da aynı girdiyi gönderin; ekranın sadece API üzerinde bir kullanım katmanı olduğunu gözlemleyin.

İkinci turda policy.py'yi ve politika testlerini okuyun. p × V - C hesabını kağıtta yapın. Sonra schemas.py üzerinden yanlış alan gönderince niçin 422 aldığınızı takip edin. Bu sıra, model matematiğine girmeden uygulamanın temel karar işini kavramayı sağlar.

Üçüncü turda veri raporunu, özellik kararlarını ve split.py'yi okuyun. features.py ve train.py ile eğitimde fit olan şeyleri işaretleyin. comparison raporunu açarak seçilen ölçüt ile iş kapasitesi ölçütünün farkını bulun. En son test raporunu okuyun; sonucu gördükten sonra hangi değişiklikleri yapmadığımızı not edin.

Günlük kullanım için PyCharm'da bir uvicorn API Run Configuration ve bir Streamlit Run Configuration yeterlidir. Ayrıntılı komutlar docs/api.md ve docs/dashboard.md içindedir. Aşama 8 için Docker/Make/CI komutları docs/delivery.md içindedir. Mevcut freeze ve test sonuçlarını silerek yeniden başlatmak günlük kullanım adımı değildir.

17. Bu projenin açık sınırları ve sonraki araştırmalar

Modelin üretimde kullanılabilirliği gösterilmedi. Kesin müşteri kimliği/tarih eksikliği, tarihsel veri, dönemler arası sınıf oranı değişimi ve kalibre edilmemiş olasılıklar önemli sınırlardır. Yaş, medeni durum ve eğitim gibi alanlar bulunduğundan gerçek uygulama öncesinde kullanım amacı, veri yetkisi, alt grup davranışı ve uygun yönetişim ayrıca değerlendirilmelidir. Bu projede alt grup adaletinin sağlandığı veya mevzuat uygunluğunun tamamlandığı iddia edilmez.

Bir sonraki modelleme deneyi için önce yeni değerlendirme tasarımı gerekir. Yeni zamansal veri, doğrulanmış karar-anı özellikleri, iş kapasitesine uygun önceden belirlenmiş seçim ölçütü ve eğitim içi uygun kalibrasyon araştırılabilir. Basit kural ve model yine aynı koşullarda karşılaştırılmalıdır. Eski test artık görüldüğü için ona tekrar tekrar bakarak geliştirme yapmak yeni bağımsız sonuç üretmez.

Gerçek kampanya etkisi için uygun kontrol gruplu bir deney tasarımı, gerçek maliyet/değer tanımları ve izleme gerekir. Üretim servisinde kimlik doğrulama, yetki, gizlilik, operasyonel izleme ve yük testleri de ayrı kapsamdır. Bunların hiçbiri yerel portföy ekranının varlığından otomatik çıkmaz.

18. Sık sorulan sorular ve mülakatta anlatım

“Bu kadar düşük AUC varsa proje başarısız mı?”

Modelin bu veride güçlü olmadığı sonucu var. Buna karşılık veri sözleşmesi, sızıntı kontrolü, şeffaf karşılaştırma, tutarlı servis, SQL ve test altyapısı tamamlanmış bir mühendislik çalışmasıdır. İyi bir portföy, yalnız yüksek sayı göstermeyi değil yanlış iddiayı önlemeyi de gösterir. Model zayıflığını saklamak yerine hangi kanıtın bunu gösterdiğini anlatın.

“Neden en çok abonelik bulan kuralı servise koymadık?”

Model seçimi ve politika testten önce donduruldu. Testte kuralın daha iyi olduğunu görünce mevcut deneyin seçimini değiştirmek, önceden verilmiş kararı sonuca göre yeniden yazmak olurdu. Gelecek bir ürün kararında kural da aday olabilir; bunun ayrı değerlendirilmesi gerekir. Bu prototip frozen modelin servislenmesini göstermek için aynı modeli korur.

“Neden duration'ı kullanmıyoruz?”

Karar aramadan önce veriliyor; görüşme süresi aramadan sonra biliniyor. Daha yüksek metrik uğruna duration kullanmak, kullanım anında olmayan bilgiyi modelin eline vermektir. Bu nedenle hem özellik listesinde yok hem HTTP/CSV girdisinde reddediliyor.

“Neden birden fazla kilit dosyası var?”

uv.lock frozen ML ortamını kaydeder. API ve UI daha sonra eklendi; önceki deneyin kaydını değiştirmeden ek paketleri ayrı hash-kilitli dosyalarda tuttuk. UI kilidi API ve ortak ML paket sürümleriyle uyumludur. Linux konteyner/CI için platforma özgü watchdog bağımlılığı ayrıca requirements-container.lock içinde tutulur. Paketlerin sadece isimleri değil tam sürümleri ve indirme özetleri de önemlidir.

“Database, API ve UI arasındaki bağlantı ne?”

DB tarihsel veri/özellik depolama katmanıdır. API, kendisine verilen kayıtları frozen modelle skorlar. UI dosya ve senaryo girdisini API'ye aktarır. API her istekte DB'den okumaz. SQL özellik sözleşmesiyle servis girdisinin uyumu ayrıca test edilmiştir.

“Yeni CSV'de lift neden yok?”

Lift hesaplamak için gerçek abonelik etiketleri ve karşılaştırılan evrenin pozitif oranı gerekir. Yeni aday dosyasında bunlar yoktur. Model tahminini gerçek sonuç yerine koyup lift hesaplamak ölçüm değil döngüsel bir iddia olur.

“Hangi becerileri gösteriyorum?”

Veri kalite kontrolü; sızıntısız ML değerlendirmesi; scikit-learn pipeline; sınırlı boosting karşılaştırması; SQL ve transactional yükleme; sıkı API tasarımı; CSV ve kullanıcı durumu yönetimi; birim/entegrasyon/akış testleri; bağımlılık/provenance yönetimi; Docker/CI hazırlığı ve dürüst teknik iletişim. İlk mesajdaki iş ilanlarının tamamının doğrulanmış beceri frekansı bu raporda çıkarılmıyor; rapor tamamlanmış projenin kanıtlarına dayanıyor.

İki dakikalık anlatım taslağı

“Bankanın sınırlı arama kapasitesi için tarihsel abonelik skorunu karar listesine dönüştürdüm. Önce hangi bilgilerin arama öncesinde bulunabileceğini tanımladım ve duration gibi sızıntı alanlarını çıkardım. Kaynak sırasına göre sabit ayrımlar oluşturdum; ön işlemeyi yalnız eğitimde fit ettim. Dummy, lojistik ve üç LightGBM adayını doğrulama AP'siyle karşılaştırıp modeli dondurdum. Testte model rastgele seçimi geçti, fakat basit geçmiş kuralını geçemedi; bunu ve düşük kalibrasyonu raporladım. SQL özellik görünümünü Python'la doğruladım, modeli FastAPI'de bir kez yükleyip sıkı şemayla servisledim. Streamlit ekranından dosya yükleme, senaryo ve CSV indirme akışını bağladım. Testler ve Docker/CI hazırlığıyla çalışmanın tekrar incelenebilir olmasını sağladım. Gerçek ROI veya üretime hazır model iddiasında bulunmadım.”

19. Kanıtlar, kaynaklar ve raporun sınırı

Bu raporun birincil dayanağı projenin kendi kodu ve kayıtlı sonuçlarıdır. Eski aşama raporlarındaki “test henüz değerlendirilmedi” gibi ifadeler, o aşamanın tarihsel durumunu anlatır. Güncel bütünsel durum bu rapor ve teslim kontrol kaydında belirtilir.

İddia	Yerel kanıt
Veri sayıları ve sütun gerekçeleri	reports/data_audit.md ve .json
Ayrımlar ve tekrar sayıları	reports/split_manifest.json
Baseline ayarları/sonuçları	reports/baseline_validation.json
Adaylar ve AP seçimi	configs/comparison.json, reports/comparison_manifest.json
Freeze testten önce kaydedildi	reports/selection_freeze.json, reports/test_release.json
Son test metrikleri	reports/test_metrics.json ve test_report.md
Kontrol akışı onarımının sınırı	reports/evaluation_runtime_revision.json
SQL/Python eşitliği	reports/database_verification.json
API ve CSV tutarlılığı	reports/api_verification.json, dashboard_verification.json
Yerel teslim kontrolleri	reports/delivery_verification.json

Dış kaynaklar: UCI Bank Marketing, veri kümesi DOI 10.24432/C5K306; Moro, Rita ve Cortez (2014), ilgili çalışma DOI 10.1016/j.dss.2014.03.001. Veri için UCI'de belirtilen CC BY 4.0 atfı korunur. Kaynak bağlantısı ve dosya edinim özeti data/README.md ile veri denetiminde kayıtlıdır.

Yöntem belgeleri: scikit-learn Pipeline, olasılık kalibrasyonu, LightGBM parametreleri, FastAPI yaşam döngüsü, Pydantic strict mode, Streamlit AppTest.

Rapor, gerçekleştirilmiş yerel iş ile gelecekte yapılabilecek işi ayırır. Uzak CI başarısı, GitHub yayını, üretim güvenliği, adalet ve gerçek kampanya finansal etkisi tamamlanmış işler olarak sunulmaz.

20. Teslim kontrol kaydı

Aşama 8'in yerel hazırlığı tamamlandı. GitHub'a push, uzak repo oluşturma ve halka açık dağıtım yapılmadı. GitHub Actions dosyası hazırlandı fakat uzak GitHub runner'ında çalıştırılmadı.

Kontrol	Gerçek sonuç
Tam yerel test takımı	PostgreSQL dahil 269 geçti; bilinen 1 TestClient deprecation uyarısı
Modelsiz/ham verisiz kaynak kopyası	261 geçti, 8 PostgreSQL testi bu çalıştırmada dışarıda; mevcut kilitli ortam kullanıldı
Ruff lint / format	34 dosya temiz
mypy	22 kaynak/uygulama/script dosyası temiz
ML/API/UI kilitleri	Mevcut dosya özetleri korundu
Docker imajı	Linux üzerinde oluşturuldu; imaj katmanında ham veri, model, rapor dizini ve .env yok
Konteyner paket uyumu	50 kurulu paket uyumlu; Linux eki dahil
Docker API / UI	Sağlıklı; yerel 18000 / 18502 portları
Gerçek frozen model	Aynı dosya özetiyle yüklendi; üç sentetik kayıtta deterministik skor
Kampanya smoke	İki kayıtlık liste; K=0 ve negatif beklenen değer için boş liste; CSV sırası korundu
GitHub yayını / uzak CI	Yapılmadı / çalıştırılmadı

Yerel Docker ekranı: http://127.0.0.1:18502. Docker API belgeleri: http://127.0.0.1:18000/docs. Önceki yerel Python çalışma akışının portları 8502/8000 olarak kalır. Konteyner içi API adresi ile tarayıcıdan açılan belge adresi ayrıdır; bu ayrım için ek ekran akış testi yazıldı.

Kullanım için proje kökünde make serve, ardından make serving-verify kullanılır. Güvenilir frozen model dosyasının önceden yerelde bulunması ve Docker'ın açık olması gerekir. make serving-down yalnız API/UI konteynerlerini durdurup kaldırır; PostgreSQL verisini silmez. Ayrıntılar docs/delivery.md içindedir.

Tamamen yeni bir bilgisayarda, ham veri ve model olmadan CI fixture testleri çalışabilir; gerçek-model ekranı kendiliğinden hazır olmaz. Bu sınır gizlenmemiştir. Modeli başka bir ortamda yeniden üretmekle aynı pickle baytlarını elde etmek aynı garanti değildir. Mevcut model paketinin sağlanması veya yeni sürüm tasarımı GitHub tesliminden önce ayrıca ele alınmalıdır.

PDF görsel olarak kontrol edildi; Türkçe karakterler, sayfa başlıkları, içindekiler, mimari şema, tablolar ve sayfa numaraları doğrulandı. Bu belge öğrenme ve inceleme amacıyla hazırlandı. Sonraki çalışma, raporu esas alarak GitHub portföy sunumuna geçmektir; bu yayın işlemi gerçekleştirilmedi.